Q – 1 Create a web page using AngularJS to take the user's name and display a customized welcome message.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-app="">

   <div ng-init="name="">

      <h2>Welcome Page</h2>

      <p>Enter Your Name:</p>
      <input type="text" ng-model="name">

      <p>Welcome {{name}}!</p>

   </div>

</body>
</html>
```

- **Output:-**



Q – 2 Create a web page using AngularJS to accept a number and display its square and cube.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

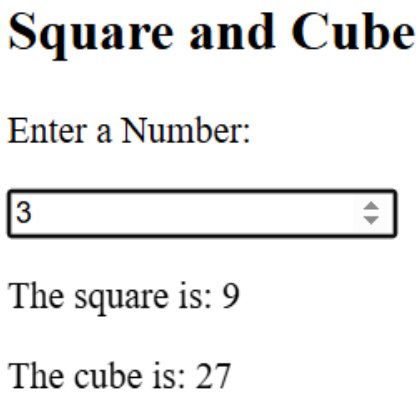
<body ng-app="">

   <div ng-init="num=0">

      <h2>Square and Cube of a Number</h2>

      <p>Enter a Number:</p>
      <input type="number" ng-model="num">
      <p>The square is: {{num * num}}</p>
      <p>The cube is: {{num * num * num}}</p>
   </div>
</body>
</html>
```

- **Output :-**



Q – 3 Create a web page using AngularJS to accept first name and last name and display the user's full name using AngularJS expressions.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-app="">

  <div>
    <h2>User's Full Name</h2>

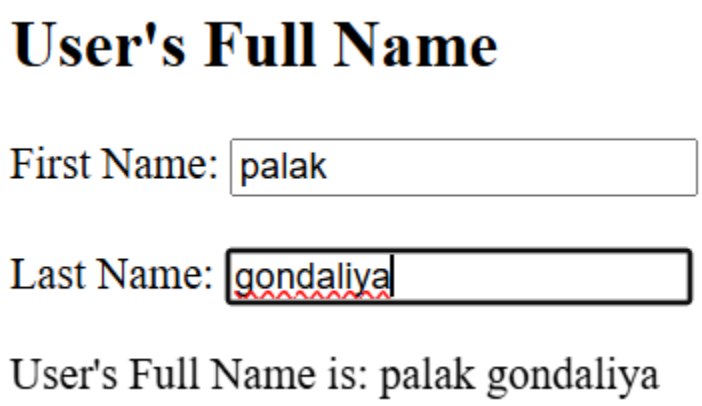
    First Name:
    <input type="text" ng-model="firstname">
    <br><br>

    Last Name:
    <input type="text" ng-model="lastname">

    <p>User's Full Name is: {{firstname + " " + lastname}}</p>
  </div>

</body>
</html>
```

- **Output :-**

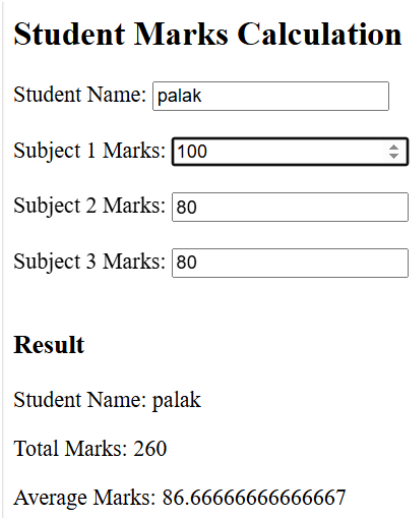


Q – 4 Create a web page using AngularJS to accept a student's marks in three subjects and calculate the total and average marks.

- **Input :-**

```
<h2>Student Marks Calculation</h2>
Student Name:
<input type="text" ng-model="studentname">
<br><br>
Subject 1 Marks:
<input type="number" ng-model="marks1">
<br><br>
Subject 2 Marks:
<input type="number" ng-model="marks2">
<br><br>
Subject 3 Marks:
<input type="number" ng-model="marks3">
<br><br>
<h3>Result</h3>
<p>Student Name: {{studentname}}</p>
<p>Total Marks:
   {{(marks1 || 0) * 1 + (marks2 || 0) * 1 + (marks3 || 0) * 1}}
</p>
<p>Average Marks:
   {{((marks1 || 0) * 1 + (marks2 || 0) * 1 + (marks3 || 0) * 1) / 3}}
</p>
```

- **Output :-**

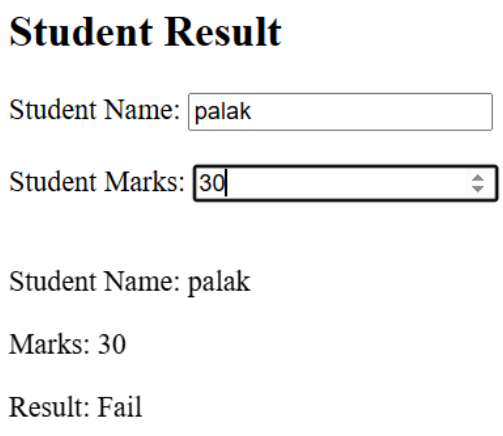


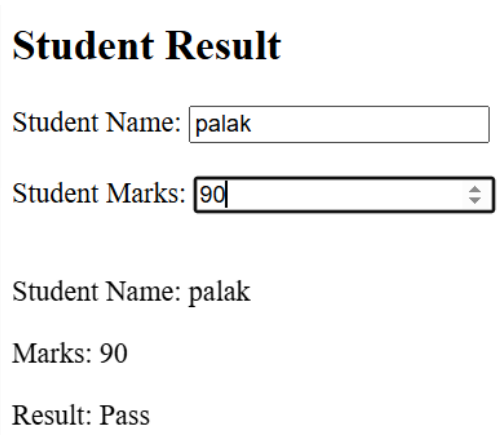
Q – 5 Create a web page using AngularJS to accept a student's name and marks and display whether the student has passed or failed.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Result</title>
   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-app="">
   <h2>Student Result</h2>
   Student Name:
   <input type="text" ng-model="studentname">
   <br><br>
   Student Marks:
   <input type="number" ng-model="marks">
   <br><br>
   <p>Student Name: {{studentname}}</p>
   <p>Marks: {{marks}}</p>
   <p>Result:
      <span ng-if="marks >= 35 && marks <= 100">Pass</span>
      <span ng-if="marks < 35 && marks >= 0">Fail</span>
   </p>
</body>
</html>
```

- **Output :-**



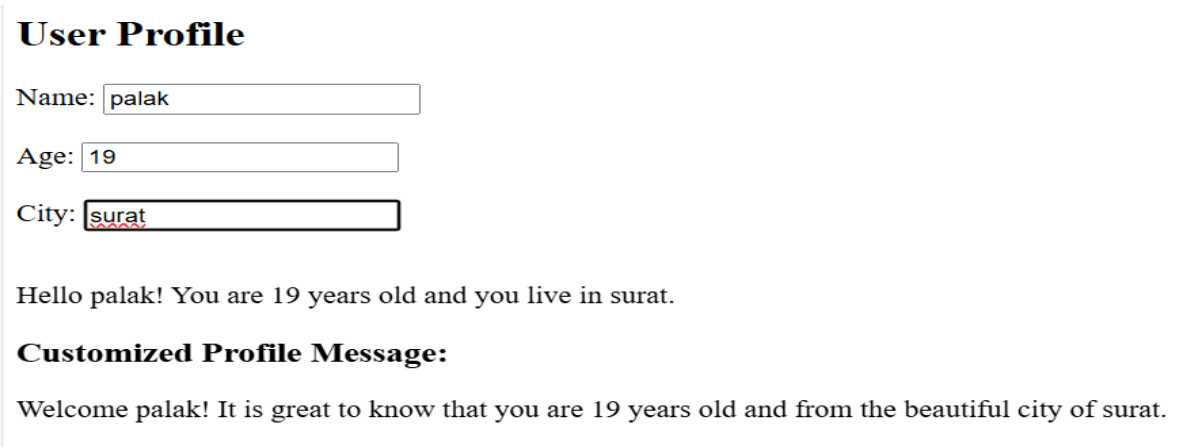


Q – 6 Create a web page using AngularJS to accept the user's name, age, and city and display a customized profile message.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-app="">
   <div>
      <h2>User Profile</h2>
      Name:
      <input type="text" ng-model="name">
      <br><br>
      Age:
      <input type="number" ng-model="age">
      <br><br>
      City:
      <input type="text" ng-model="city">
      <p>Hello {{name}}! You are {{age}} years old and you live in {{city}} </p>
      <h3>Customized Profile Message:</h3>
      <p> Welcome {{name}}! It is great to know that you ar  {{age}} years old and from
the beautiful city of {{city}}</p>
   </div>
</body>
</html>
```

- **Output :-**



Q – 7 Create a web page using AngularJS to accept a number and display its multiplication table.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
   <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>

<body ng-app="myApp" ng-controller="TableController">

   <h2>Multiplication Table</h2>

   Enter a Number:
   <input type="number" ng-model="num">

   <br><br>

   <button ng-click="generateTable()">
      Generate Table
   </button>

   <br><br>

   <div ng-if="tableData.length > 0">
      <h3>Table for {{num}}:</h3>
      <p ng-repeat="row in tableData">
         {{row}}
      </p>
   </div>

   <script>
      var app = angular.module('myApp', []);

      app.controller('TableController', function($scope) {
         $scope.tableData = [];

         $scope.generateTable = function() {
            $scope.tableData = [];

            if ($scope.num !== undefined && $scope.num !== null) {
```

```
            for (var i = 1; i <= 10; i++) {
                var result = $scope.num + " × " + i + " = " + ($scope.num * i);
                $scope.tableData.push(result);
            }
        }
    };
});
</script>

</body>
</html>
```

- **Output :-**

Multiplication Table

Enter a Number: 10

Generate Table

Table for 10:

10 × 1 = 10

10 × 2 = 20

10 × 3 = 30

10 × 4 = 40

10 × 5 = 50

10 × 6 = 60

10 × 7 = 70

10 × 8 = 80

10 × 9 = 90

10 × 10 = 100

Q – 8 Create a custom <user-card> directive that displays a user's name, email, and profile image.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    .user-card {
      width: 220px;
      padding: 16px;
      border: 1px solid #ccc;
      border-radius: 8px;
      text-align: center;
      font-family: sans-serif;
      box-shadow: 2px 2px 8px rgba(0,0,0,0.1);
    }
    .user-card img {
      width: 100px;
      height: 100px;
      border-radius: 50%;
      object-fit: cover;
      margin-bottom: 10px;
    }
  </style>
</head>

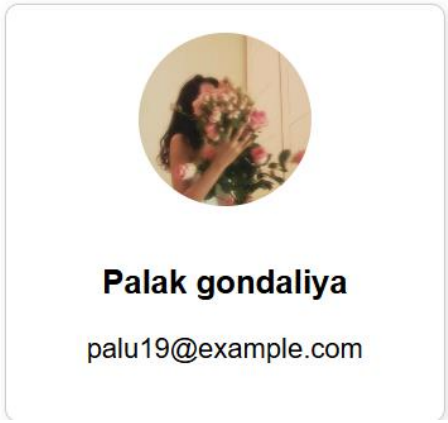
<body ng-app="myApp" ng-controller="myCtrl">
  <h2>User Profile</h2>
  <user-card
    name="{{user.name}}"
    email="{{user.email}}"
    image="{{user.image}}">
  </user-card>
  <script>
    var app = angular.module("myApp", []);
    // Controller providing sample data
    app.controller("myCtrl", function($scope) {
```

```
        $scope.user = {
            name: "Palak gondaliya ",
            email: "palu19@example.com",
            image: "user.jpg"
        };
    });
    // Custom Directive Definition
    app.directive("userCard", function() {
        return {
            restrict: "E", // E = Element (<user-card>)
            scope: {
                name: "@",
                email: "@",
                image: "@"
            },
            template: `
                <div class="user-card">
                    <img ng-src="{{image}}" alt="{{name}}'s photo">
                    <h3>{{name}}</h3>
                    <p>{{email}}</p>
                </div>          };

    });
</script>

</body>
</html>
```

- **Output :-**



Q-9 Create a reusable student-card directive that:

• Receives student information using =.

• Displays the student's name, course, and marks.

• Calculates pass/fail.

• Has a View Details button.

• Calls a controller function using &.

- **Input :-**

```
<!DOCTYPE html>
<html>
<head>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    .student-card {
      width: 220px;
      padding: 15px;
      margin: 10px;
      border: 2px solid #333;
      border-radius: 8px;
      display: inline-block;
      font-family: sans-serif;
      background-color: #fcfcfc;
    }
    .pass { color: green; font-weight: bold; }
    .fail { color: red; font-weight: bold; }
  </style>
</head>

<body ng-app="myApp" ng-controller="myCtrl">

  <h2>Student List</h2>

  <!-- Reusable Directive with '=' for data and '&' for callback -->
  <student-card
    ng-repeat="s in students"
    info="s"
```

```
        on-view="showDetails(selected)">
    </student-card>

    <script>
        var app = angular.module("myApp", []);

        app.controller("myCtrl", function($scope) {
            $scope.students = [
                { name: "Palak Gondaliya ", course: "BCA", marks: 75 },
                { name: "Ritu amipara", course: "IT", marks: 32 },
                { name: "Mirali Patel", course: "Diploma", marks: 58 }
            ];

            // Function triggered via directive
            $scope.showDetails = function(student) {
                alert("Details:\nName: " + student.name + "\nCourse: " + student.course +
"\nMarks: " + student.marks);
            };
        });

        app.directive("studentCard", function() {
            return {
                restrict: "E",
                scope: {
                    info: "=",     // Two-way binding for student object
                    onView: "&"    // Method binding for controller callback
                },
                template: `
                    <div class="student-card">
                        <h3>{{info.name}}</h3>
                        <p><b>Course:</b> {{info.course}}</p>
                        <p><b>Marks:</b> {{info.marks}}</p>
                        <p><b>Status:</b>
                            <span ng-class="info.marks >= 40 ? 'pass' : 'fail'">
                                {{info.marks >= 40 ? 'Pass' : 'Fail'}}
                            </span>
                        </p>
                        <button ng-click="onView({ selected: info })">View Details</button>
                    </div>
                `
```

```
        };
    });
</script>

</body>
</html>
```

- **Output :-**

Student List

Palak Gondaliya
Course: BCA
Marks: 75
Status: Pass
View Details

Ritu amipara
Course: IT
Marks: 32
Status: Fail
View Details

Mirali Patel
Course: Diploma
Marks: 58
Status: Pass
View Details

127.0.0.1:5500 says

Details:
Name: Palak Gondaliya
Course: BCA
Marks: 75

OK

Q-10 Develop a simple AngularJS application that contains a user registration form with the following requirements:

1. Accept Name, Email, and Age from the user.

2. Apply form validation:

o All fields are required.

o Name must contain at least 3 characters.

o Enter a valid email address.

o Age must be between 18 and 60.

3. Display appropriate validation error messages when the input is invalid.

4. On successful form submission, display a welcome message containing the user's name and entered details.

5. Prevent form submission until all validation requirements are satisfied.

- **Input :-**

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">

<head>
   <meta charset="UTF-8">
   <title>AngularJS Registration Form</title>

   <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>

   <style>
      body {
         font-family: Arial, sans-serif;
         background-color: #f5f5f5;
         padding: 30px;
      }

      .form-container {
         max-width: 500px;
```

```
    background-color: #bdcced;
    padding: 25px 30px;
    margin: auto;
    border-radius: 10px;
    box-shadow: 0px 5px 15px rgba(0, 0, 0, 0.2);
}

h2 {
    text-align: center;
    margin-bottom: 20px;
    color: #333;
}

label {
    font-weight: bold;
    display: block;
    margin-top: 15px;
    margin-bottom: 5px;
    color: #0a1254;
}

input[type="text"],
input[type="email"],
input[type="number"] {
    width: 100%;
    padding: 8px 10px;
    border: 2px solid #012143;
    border-radius: 10px;
    box-sizing: border-box;
    background-color: #d1d9eb;
}

.error {
    color: red;
    font-size: 14px;
    margin-top: 5px;
}

button {
    display: block;
```

```
    width: 100%;
    padding: 10px;
    margin-top: 20px;
    border: none;
    background-color: #013369;
    color: #fff;
    font-size: 25px;
    border-radius: 10px;
    cursor: pointer;
}

button:hover {
    background-color: #014894;
}

button:disabled {
    background-color: gray;
    cursor: not-allowed;
}

hr {
    margin: 25px 0;
}

.data-preview {
    background-color: #d0daf2;
    padding: 15px;
    border-radius: 10px;
    border: 4px solid #0a0446;
}

.data-preview p {
    margin: 5px 0;
    color: #0a0446;
}

.success {
    color: green;
    text-align: center;
}
```

```
    </style>
</head>

<body>

<div class="form-container" ng-controller="formCtrl">

    <h2>User Registration</h2>

    <form name="regForm" ng-submit="submitForm()" novalidate>

        <!-- Name -->
        <label>Name:</label>
        <input type="text"
               name="name"
               ng-model="name"
               placeholder="Enter your name"
               ng-minlength="3"
               required>

        <div class="error"
             ng-if="regForm.name.$touched || regForm.$submitted">
            <span ng-if="regForm.name.$error.required">
                Name is required.
            </span>
            <span ng-if="regForm.name.$error.minlength">
                Name must contain at least 3 characters.
            </span>
        </div>

        <!-- Email -->
        <label>Email:</label>
        <input type="email"
               name="email"
               ng-model="email"
               placeholder="Enter email"
               required>

        <div class="error"
             ng-if="regForm.email.$touched || regForm.$submitted">
```

```
        <span ng-if="regForm.email.$error.required">
            Email is required.
        </span>
        <span ng-if="regForm.email.$error.email">
            Please enter a valid email address.
        </span>
    </div>

    <!-- Age -->
    <label>Age:</label>
    <input type="number"
           name="age"
           ng-model="age"
           placeholder="Enter your age"
           min="18"
           max="60"
           required>

    <div class="error"
         ng-if="regForm.age.$touched || regForm.$submitted">
        <span ng-if="regForm.age.$error.required">
            Age is required.
        </span>
        <span ng-if="regForm.age.$error.min ||
                     regForm.age.$error.max">
            Age must be between 18 and 60.
        </span>
        <span ng-if="regForm.age.$error.number">
            Please enter a valid age.
        </span>
    </div>

    <!-- Submit Button -->
    <button type="submit" ng-disabled="regForm.$invalid">
        SUBMIT
    </button>
</form>
<!-- Form Data Preview -->
<div class="data-preview">
```

```
        <h3>Form Data Preview</h3>

        <p>
            <strong>Name:</strong>
            {{ name }}
        </p>
        <p>
            <strong>Email:</strong>
            {{ email }}
        </p>
        <p>
            <strong>Age:</strong>
            {{ age }}
        </p>
    </div>
    <div ng-if="submitted">
        <hr>
        <h2 class="success">
            Welcome, {{ name }}!
        </h2>
        <div class="data-preview">
            <h3>Registration Successful</h3>
            <p><strong>Name:</strong> {{ name }}</p>
            <p><strong>Email:</strong> {{ email }}</p>
            <p><strong>Age:</strong> {{ age }}</p>
        </div>

    </div>

</div>

<script>
    var app = angular.module("myApp", []);

    app.controller("formCtrl", function ($scope) {

        $scope.name = "";
        $scope.email = "";
        $scope.age = null;
        $scope.submitted = false;
```

```
        $scope.submitForm = function () {

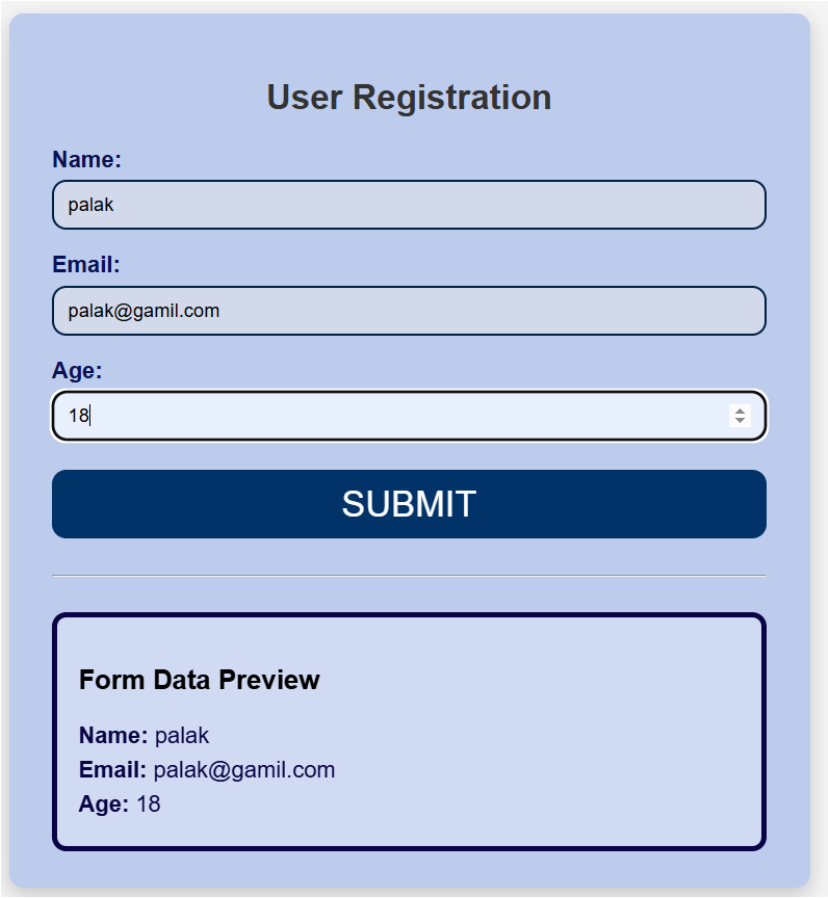
            if ($scope.regForm.$valid) {
                $scope.submitted = true;
            }

        };

    });
</script>

</body>
</html>
```

- **Output :-**



Q-11 create a Simple Task Manager (To-Do App) web application that allows :

i) Add a Task

ii) View All Tasks

iii) Mark as Completed

iv) Delete Task Project Structure

- **Input (index.html):-**

```
<!DOCTYPE html>
<html lang="en">

<head>
   <title>Simple To-do App</title>

   <link rel="stylesheet" href="style.css">

   <!-- AngularJS -->
   <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>

   <script src="app.js"></script>
</head>

<body ng-app="taskApp" ng-controller="TaskController">

   <div class="container">

      <h1>Simple To-do APP</h1>

      <!-- Add Task -->
      <div class="add-task">

         <input type="text" ng-model="newTask" placeholder="Enter a task">

         <button ng-click="addTask()">
            Add Task
         </button>

      </div>
```

```
<!-- View All Tasks -->
<h2>All Tasks</h2>

<p ng-if="tasks.length === 0">
   No tasks available.
</p>

<div class="task" ng-repeat="task in tasks" ng-class="{completed: task.completed}">

   <span>
      {{ task.title }}
   </span>

   <div>

      <!-- Mark Completed -->
      <button class="complete" ng-click="completeTask(task)">
         {{ task.completed ? 'Undo' : 'Complete' }}
      </button>

      <!-- Delete -->
      <button class="delete" ng-click="deleteTask($index)">
         Delete
      </button>

   </div>

</div>

</body>

</html>
```

- **Input (style.css):-**
- { box-sizing: border-box; }

```
body { font-family: Arial, Helvetica, sans-serif; margin: 0; min-height: 100vh;

background: linear-gradient(135deg,
    #667eea,
    #764ba2);
```

```
display: flex;
justify-content: center;
align-items: flex-start;
padding: 50px 20px;


}

.container { width: 100%; max-width: 600px; padding: 30px; background: rgba(255, 255, 255, 0.95); border-radius: 20px; box-shadow:0 20px 40px rgba(0, 0, 0, 0.2); animation: slideDown 0.6s ease; }

h1 { text-align: center; margin-top: 0; margin-bottom: 30px; font-size: 32px; color: #333; letter-spacing: 1px; }

h1::after { content: ""; display: block; width: 60px; height: 4px; margin: 10px auto 0; border-radius: 10px; background: linear-gradient(90deg, #667eea, #764ba2); }

.add-task { display: flex; gap: 12px; margin-bottom: 30px; }

input { flex: 1; padding: 13px 15px; font-size: 15px; border: 2px solid #e0e0e0; border-radius: 10px; outline: none; background: #f9f9f9; transition: all 0.3s ease; }

input:focus { border-color: #667eea; background: white; box-shadow: 0 0 0 4px rgba(102, 126, 234, 0.15); }

input::placeholder { color: #999; }

button { padding: 11px 16px; border: none; border-radius: 10px; cursor: pointer; color: white; font-size: 14px; font-weight: bold; transition: transform 0.2s ease, box-shadow 0.2s ease, background 0.2s ease; }

button:active { transform: scale(0.95); }

.add-task button { background: linear-gradient(135deg, #667eea, #764ba2);

box-shadow:
   0 5px 15px rgba(102, 126, 234, 0.3);


}

.add-task button:hover { transform: translateY(-2px);
```

```
box-shadow:
    0 8px 20px rgba(102, 126, 234, 0.4);


}

h2 { color: #444;

font-size: 21px;

margin-bottom: 15px;


}

p { text-align: center;

color: #888;

padding: 20px;

background: #f8f8f8;

border-radius: 10px;


}

.task { display: flex;

justify-content: space-between;

align-items: center;

gap: 15px;

padding: 15px 18px;

margin-top: 12px;

background: #ffffff;
```

```
border: 1px solid #eeeeee;

border-radius: 12px;

box-shadow:
   0 4px 10px rgba(0, 0, 0, 0.06);

transition:
   transform 0.25s ease,
   box-shadow 0.25s ease,
   border-color 0.25s ease;


}

.task:hover { transform: translateY(-3px);

border-color: #667eea;

box-shadow:
   0 8px 20px rgba(0, 0, 0, 0.12);


}

.task span { flex: 1;

font-size: 16px;

color: #333;

word-break: break-word;

transition: color 0.3s ease;


}

.complete { background: linear-gradient(135deg, #28a745, #20c997);
```

```
box-shadow:
   0 4px 10px rgba(40, 167, 69, 0.25);


}

.complete:hover { transform: translateY(-2px);

box-shadow:
   0 7px 15px rgba(40, 167, 69, 0.35);


}

.delete { background: linear-gradient(135deg, #ff416c, #ff4b2b);

box-shadow:
   0 4px 10px rgba(255, 65, 108, 0.25);


}

.delete:hover { transform: translateY(-2px);

box-shadow:
   0 7px 15px rgba(255, 65, 108, 0.35);

}

.completed { background: #f0fff4;

border-color: #b7ebc6;

}

.completed span { text-decoration: line-through;

color: #999;
opacity: 0.8


}

@keyframes slideDown {
```

```
from {
    opacity: 0;
    transform: translateY(-30px);
}
to {
    opacity: 1;
    transform: translateY(0);
}

@media (max-width: 600px) {

body {
    padding: 20px 10px;
}

.container {
    padding: 20px;

    border-radius: 15px;
}

h1 {
    font-size: 26px;
}
.add-task {
    flex-direction: column;
}
.add-task button {
    width: 100%;
}
.task {
    flex-direction: column;

    align-items: stretch;
}
.task span {
    margin-bottom: 8px;
}
.task div {
    display: flex
    gap: 8px;
```

```
}
.task button {
    flex: 1;
}

}
```

- **Input (app.js):-**

```
var app = angular.module("taskApp", []);

app.controller("TaskController", function ($scope) {

    // Store all tasks
    $scope.tasks = [];

    // New task entered by user
    $scope.newTask = "";

    // Add Task
    $scope.addTask = function () {

        if ($scope.newTask.trim() === "") {
            return;
        }

        var task = {
            title: $scope.newTask,
            completed: false
        };

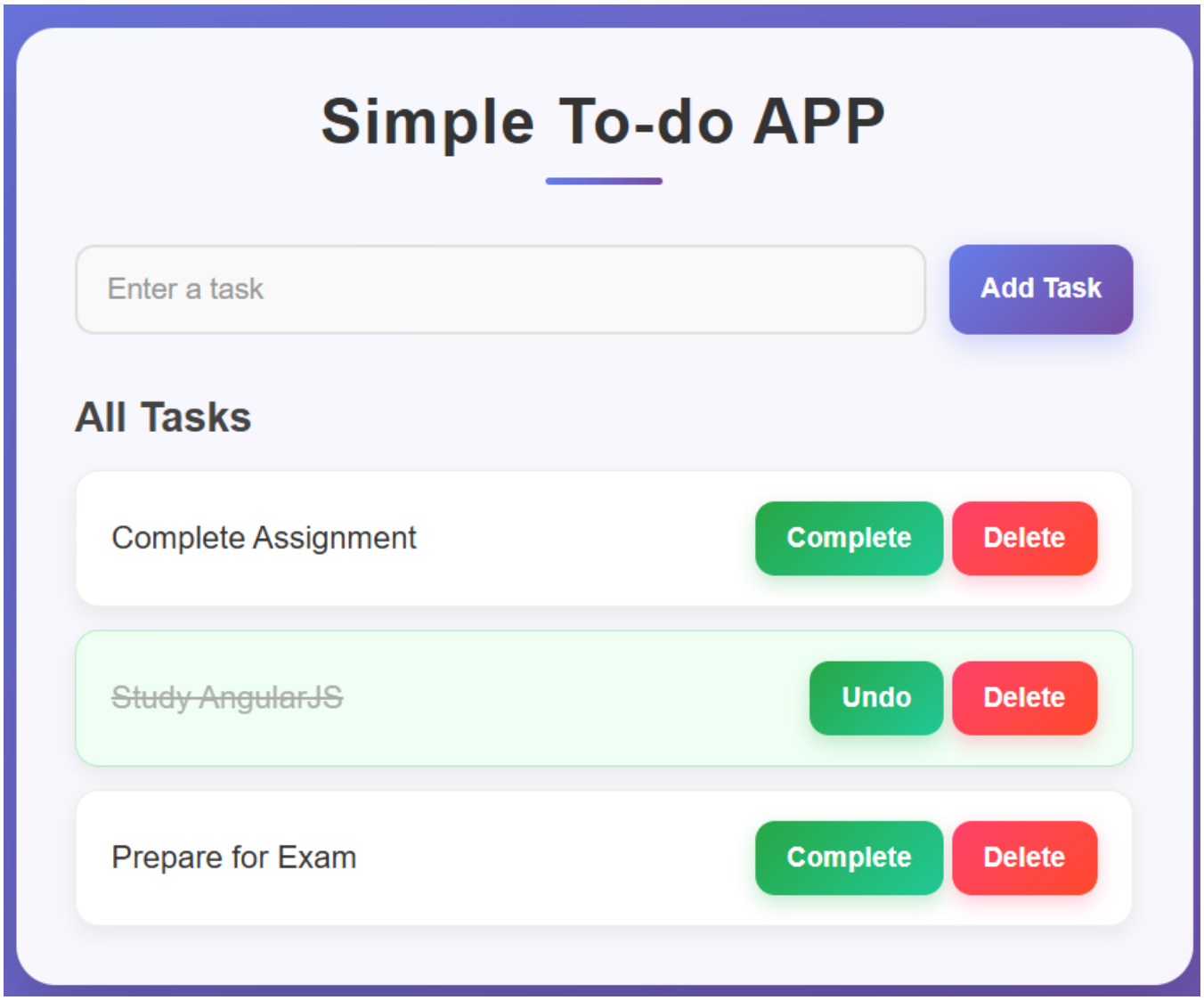
        $scope.tasks.push(task);
        // Clear input
        $scope.newTask = "";
    };
    // Mark task as completed
    $scope.completeTask = function (task) {
        task.completed = !task.completed;

    };
    $scope.deleteTask = function (index) {
```

```
        $scope.tasks.splice(index, 1);    };

    });
```

- **Output :-**



Q-12 Employee Management System

Employee(employeeId, name, department, designation, email, salary)

1. Implement GET – Retrieve all employees.

2. Implement DELETE – Remove an employee by employee ID.

3. Use MongoDB for storage.

4. Create AngularJS interface with a delete button for each employee.

- **Input(server.js):-**

```
const express = require("express");
const mongoose = require("mongoose");
const path = require("path");

const app = express();

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/EmployeeDB")
    .then(() => console.log("MongoDB Connected"))
    .catch(err => console.log(err));

// Employee Schema
const employeeSchema = new mongoose.Schema({
```

```
    employeeId: {
        type: Number,
        required: true,
        unique: true
    },
    name: String,
    department: String,
    designation: String,
    email: String,
    salary: Number
});

// Employee Model
const Employee = mongoose.model("Employee", employeeSchema);

// GET: Retrieve all employees
app.get("/api/employees", async (req, res) => {
    try {
        const employees = await Employee.find();
        res.json(employees);
    } catch (err) {
        res.status(500).json({
            message: "Error retrieving employees"
        });
    }
});
```

```
// DELETE: Remove employee by employee ID
app.delete("/api/employees/:employeeId", async (req, res) => {
    try {
        const employeeId = Number(req.params.employeeId);

        if (!Number.isInteger(employeeId)) {
            return res.status(400).json({
                message: "Invalid employee ID"
            });
        }

        const employee = await Employee.findOneAndDelete({
            employeeId: employeeId
        });

        if (!employee) {
            return res.status(404).json({
                message: "Employee not found"
            });
        }

        res.json({
            message: "Employee deleted successfully"
        });

    } catch (err) {
        res.status(500).json({
```

```
            message: "Error deleting employee"
        });
    }
});

// Start server after MongoDB connects
mongoose.connection.once("open", () => {
    app.listen(3000, () => {
        console.log("Server running at http://localhost:3000");
    });
});
```

- **Input(public/index.html) :-**

```
<!DOCTYPE html>
<html ng-app="employeeApp">

<head>
    <title>Employee Management System</title>

    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

    <link rel="stylesheet" href="style.css">
</head>

<body ng-controller="employeeController">

    <div class="container">
```

```
<h1>Employee Management System</h1>

<button ng-click="getEmployees()">
   Refresh Employees
</button>

<p ng-if="errorMessage" class="error">
   {{ errorMessage }}
</p>

<table>
   <thead>
      <tr>
         <th>Employee ID</th>
         <th>Name</th>
         <th>Department</th>
         <th>Designation</th>
         <th>Email</th>
         <th>Salary</th>
         <th>Action</th>
      </tr>
   </thead>

   <tbody>
      <tr ng-repeat="emp in employees">
         <td>{{ emp.employeeId }}</td>
```

```
            <td>{{ emp.name }}</td>
            <td>{{ emp.department }}</td>
            <td>{{ emp.designation }}</td>
            <td>{{ emp.email }}</td>
            <td>{{ emp.salary }}</td>

            <td>
              <button class="delete-btn"
                      ng-click="deleteEmployee(emp.employeeId)">
                Delete
              </button>
            </td>
          </tr>

          <tr ng-if="employees.length === 0">
            <td colspan="7">No employees found.</td>
          </tr>
        </tbody>
      </table>

    </div>

    <script src="app.js"></script>

</body>
</html>
```

- **Input(public/app.js):-**

```
var app = angular.module("employeeApp", []);

app.controller("employeeController", function($scope, $http) {

    $scope.employees = [];
    $scope.errorMessage = "";

    // GET: Retrieve all employees
    $scope.getEmployees = function() {

        $http.get("/api/employees")
            .then(function(response) {

                $scope.employees = response.data;
                $scope.errorMessage = "";

            })
            .catch(function() {

                $scope.errorMessage =
                    "Unable to retrieve employees.";

            });
    };
```

```
// DELETE: Remove employee by ID
$scope.deleteEmployee = function(employeeId) {

    if (!confirm("Are you sure you want to delete this employee?")) {
        return;
    }

    $http.delete("/api/employees/" + employeeId)
        .then(function(response) {

            alert(response.data.message);

            // Refresh the employee list
            $scope.getEmployees();

        })
        .catch(function(error) {

            $scope.errorMessage =
                error.data && error.data.message
                    ? error.data.message
                    : "Unable to delete employee.";

        });
};

// Load employees when the page opens
```

```
    $scope.getEmployees();


});
```

- **Input(public/style.css) :-**

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 20px;
}

.container {
    max-width: 1200px;
    margin: auto;
    background-color: white;
    padding: 20px;
    border-radius: 8px;
}
h1 {
    text-align: center;
}
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}
th, td {
```

```
    border: 1px solid #cccccc;
    padding: 10px;
    text-align: left;
}
th {
    background-color: #dddddd;
}

button {
    padding: 8px 12px;
    cursor: pointer;
}
.delete-btn {
    background-color: #dc3545;
    color: white;
    border: none;
    border-radius: 4px;
}
.error {
    color: red;
}
```

- **Output :-**



Q-13 Course Registration System

Course(courseId, courseCode, courseName, credits, capacity)

1. Implement GET – List all courses (course code, title, credits).

2. Implement POST – Add a new course.

3. Implement PUT – Update course capacity or credits.

4. Use MongoDB for storage.

5. Create AngularJS form to add and edit course details.

- **Input (server.js):-**

```
const express = require("express");
const mongoose = require("mongoose");
const path = require("path");

const app = express();

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/CourseDB")
```

```
    .then(() => console.log("MongoDB Connected"))
    .catch(err => console.log("MongoDB Error:", err));

// Course Schema
const courseSchema = new mongoose.Schema({
    courseId: {
        type: Number,
        required: true,
        unique: true
    },
    courseCode: {
        type: String,
        required: true
    },
    courseName: {
        type: String,
        required: true
    },
    credits: {
        type: Number,
        required: true,
        min: 1
    },
    capacity: {
        type: Number,
        required: true,
        min: 1
```

```
    }
});

// Course Model
const Course = mongoose.model("Course", courseSchema);

// GET: List all courses
app.get("/api/courses", async (req, res) => {
    try {
        const courses = await Course.find()
            .select("courseId courseCode courseName credits capacity -_id");

        res.json(courses);
    } catch (err) {
        res.status(500).json({
            message: "Error retrieving courses"
        });
    }
});

// POST: Add a new course
app.post("/api/courses", async (req, res) => {
    try {
        const { courseId, courseCode, courseName, credits, capacity } = req.body;

        if (
            !Number.isInteger(courseId) ||
```

```
    !courseCode?.trim() ||
    !courseName?.trim() ||
    !Number.isInteger(credits) ||
    credits < 1 ||
    !Number.isInteger(capacity) ||
    capacity < 1
) {
    return res.status(400).json({
        message: "Enter valid course details"
    });
}

const course = new Course({
    courseId,
    courseCode: courseCode.trim(),
    courseName: courseName.trim(),
    credits,
    capacity
});

await course.save();

res.status(201).json({
    message: "Course added successfully",
    course
});
```

```
    } catch (err) {
        if (err.code === 11000) {
            return res.status(409).json({
                message: "Course ID already exists"
            });
        }

        res.status(500).json({
            message: "Error adding course"
        });
    }
});

// PUT: Update course capacity or credits
app.put("/api/courses/:courseId", async (req, res) => {
    try {
        const courseId = Number(req.params.courseId);
        const updates = {};

        if (!Number.isInteger(courseId)) {
            return res.status(400).json({
                message: "Invalid course ID"
            });
        }

        if (req.body.credits !== undefined) {
            if (!Number.isInteger(req.body.credits) ||
```

```
        req.body.credits < 1) {
        return res.status(400).json({
            message: "Credits must be a positive integer"
        });
    }

    updates.credits = req.body.credits;
}

if (req.body.capacity !== undefined) {
    if (!Number.isInteger(req.body.capacity) ||
        req.body.capacity < 1) {
        return res.status(400).json({
            message: "Capacity must be a positive integer"
        });
    }

    updates.capacity = req.body.capacity;
}

if (Object.keys(updates).length === 0) {
    return res.status(400).json({
        message: "Provide credits or capacity to update"
    });
}

const course = await Course.findOneAndUpdate(
```

```
            { courseId: courseId },
            { $set: updates },
            { new: true, runValidators: true }
        );

        if (!course) {
            return res.status(404).json({
                message: "Course not found"
            });
        }

        res.json({
            message: "Course updated successfully",
            course
        });

    } catch (err) {
        res.status(500).json({
            message: "Error updating course"
        });
    }
});

// Start server after MongoDB connects
mongoose.connection.once("open", () => {
    app.listen(3000, () => {
        console.log("Server running at http://localhost:3000");
```

```
    });
});
```

- **Input(public/index.html):-**

```
<!DOCTYPE html>
<html ng-app="courseApp">

<head>
    <title>Course Registration System</title>

    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

    <link rel="stylesheet" href="style.css">
</head>

<body ng-controller="courseController">

    <div class="container">

        <h1>Course Registration System</h1>

        <!-- Add and Edit Course Form -->
        <h2>{{ isEditing ? 'Edit Course' : 'Add New Course' }}</h2>

        <form ng-submit="saveCourse()" novalidate>

            <label>Course ID:</label>
            <input type="number"
```

```
        ng-model="form.courseId"
        ng-disabled="isEditing"
        required>

<label>Course Code:</label>
<input type="text"
        ng-model="form.courseCode"
        ng-disabled="isEditing"
        required>

<label>Course Name:</label>
<input type="text"
        ng-model="form.courseName"
        ng-disabled="isEditing"
        required>

<label>Credits:</label>
<input type="number"
        ng-model="form.credits"
        min="1"
        step="1"
        required>

<label>Capacity:</label>
<input type="number"
        ng-model="form.capacity"
        min="1"
```

```
            step="1"
            required>

    <button type="submit">
        {{ isEditing ? 'Update Course' : 'Add Course' }}
    </button>

    <button type="button"
            ng-if="isEditing"
            ng-click="cancelEdit()">
        Cancel
    </button>

</form>

<p class="message" ng-if="message">
    {{ message }}
</p>

<p class="error" ng-if="errorMessage">
    {{ errorMessage }}
</p>

<!-- Course List -->
<h2>All Courses</h2>

<button ng-click="getCourses()">
```

```
    Refresh Courses
</button>

<table>
    <thead>
        <tr>
            <th>Course ID</th>
            <th>Course Code</th>
            <th>Course Name</th>
            <th>Credits</th>
            <th>Capacity</th>
            <th>Action</th>
        </tr>
    </thead>

    <tbody>
        <tr ng-repeat="course in courses">
            <td>{{ course.courseId }}</td>
            <td>{{ course.courseCode }}</td>
            <td>{{ course.courseName }}</td>
            <td>{{ course.credits }}</td>
            <td>{{ course.capacity }}</td>

            <td>
                <button ng-click="editCourse(course)">
                    Edit
                </button>
```

```
            </td>
          </tr>

          <tr ng-if="courses.length === 0">
            <td colspan="6">No courses found.</td>
          </tr>
        </tbody>
      </table>

  </div>

  <script src="app.js"></script>

</body>
</html>
```

- **Input(public/app.js):-**

```
var app = angular.module("courseApp", []);

app.controller("courseController", function($scope, $http) {

  $scope.courses = [];
  $scope.form = {};
  $scope.isEditing = false;
  $scope.message = "";
  $scope.errorMessage = "";

  // GET: Retrieve all courses
```

```
$scope.getCourses = function() {

    $http.get("/api/courses")
        .then(function(response) {
            $scope.courses = response.data;
            $scope.errorMessage = "";
        })
        .catch(function() {
            $scope.errorMessage = "Unable to load courses.";
        });
};

// POST: Add course or PUT: Update course
$scope.saveCourse = function() {

    $scope.message = "";
    $scope.errorMessage = "";

    if (
        !$scope.form.courseCode ||
        !$scope.form.courseName ||
        !Number.isInteger($scope.form.courseId) ||
        !Number.isInteger($scope.form.credits) ||
        !Number.isInteger($scope.form.capacity) ||
        $scope.form.credits < 1 ||
        $scope.form.capacity < 1 ||
        $scope.form.courseId < 1
```

```
) {
    $scope.errorMessage =
        "Enter valid course details. Numeric values must be positive integers.";
    return;
}

if ($scope.isEditing) {

    // PUT: Update existing course
    var courseId = $scope.form.courseId;

    $http.put("/api/courses/" + courseId, {
        credits: $scope.form.credits,
        capacity: $scope.form.capacity
    })
    .then(function(response) {
        $scope.message = response.data.message;
        $scope.resetForm();
        $scope.getCourses();
    })
    .catch(function(error) {
        $scope.errorMessage =
            error.data && error.data.message
                ? error.data.message
                : "Unable to update course.";
    });
```

```
    } else {

        // POST: Add new course
        $http.post("/api/courses", $scope.form)
            .then(function(response) {
                $scope.message = response.data.message;
                $scope.resetForm();
                $scope.getCourses();
            })
            .catch(function(error) {
                $scope.errorMessage =
                    error.data && error.data.message
                        ? error.data.message
                        : "Unable to add course.";
            });

    }
};

// Fill the form for editing
$scope.editCourse = function(course) {

    $scope.form = angular.copy(course);
    $scope.isEditing = true;
    $scope.message = "";
    $scope.errorMessage = "";
```

```
    };
    // Cancel editing
    $scope.cancelEdit = function() {
        $scope.resetForm();
    };
    // Reset form
    $scope.resetForm = function() {
        $scope.form = {};
        $scope.isEditing = false;
    };
    // Load courses when page opens
    $scope.getCourses();
});
```

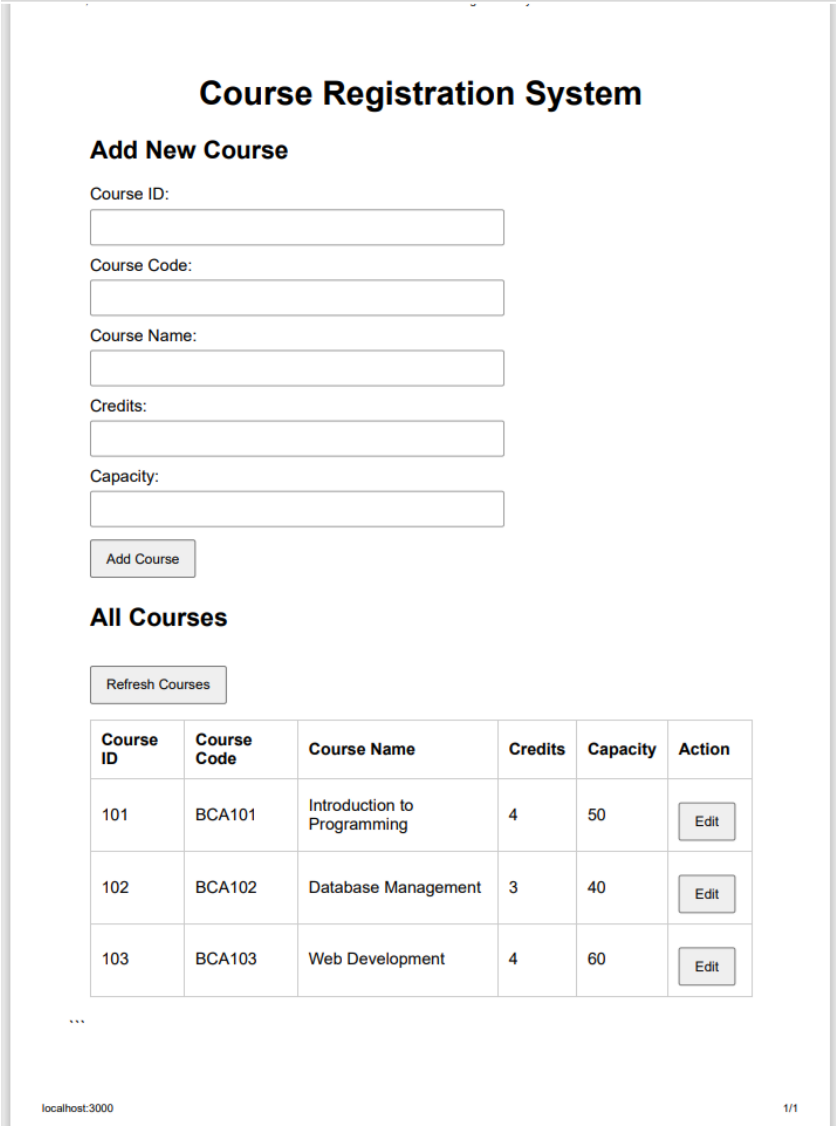
- **Input(public/style.css):-**

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 20px;
}
.container {
    max-width: 1000px;
    margin: auto;
    background-color: white;
    padding: 20px;
    border-radius: 8px;
}
```

```
h1 {
    text-align: center;
}
form {
    max-width: 400px;
    margin-bottom: 25px;
}
label {
    display: block;
    margin-top: 10px;
}
input {
    width: 100%;
    padding: 8px;
    margin-top: 5px;
    box-sizing: border-box;
}
button {
    padding: 9px 14px;
    margin-top: 12px;
    margin-right: 5px;
    cursor: pointer;
}
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
```

```
}
th, td {
    border: 1px solid #ccc;
    padding: 10px;
    text-align: left;
}
th {
    background-color: #ddd;
}
.message {
    color: green;
}
.error {
    color: red;
}
```

- **Output :-**



Q-14 Event Management System

Event(eventId, title, venue, date, fees, organizerName)

1. Implement GET – Retrieve all events (title, venue, date,fees).

2. Implement POST – Create a new event.

3. Implement DELETE – Cancel an event by event ID.

4. Use MongoDB for storage.

5. Create AngularJS form to add and cancel events.

- **Input(server.js) :-**

```
const express = require("express");
const mongoose = require("mongoose");
const path = require("path");

const app = express();

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/EventDB")
    .then(() => console.log("MongoDB Connected"))
    .catch(err => console.log("MongoDB Error:", err));

// Event Schema
const eventSchema = new mongoose.Schema({
    eventId: {
        type: Number,
        required: true,
        unique: true
    },
    title: {
        type: String,
        required: true
    },
    venue: {
        type: String,
```

```
        required: true
    },
    date: {
        type: Date,
        required: true
    },
    fees: {
        type: Number,
        required: true,
        min: 0
    },
    organizerName: {
        type: String,
        required: true
    }
});

// Event Model
const Event = mongoose.model("Event", eventSchema);

// GET: Retrieve all events
app.get("/api/events", async (req, res) => {
    try {
        const events = await Event.find().sort({ eventId: 1 });
        res.json(events);
    } catch (err) {
        res.status(500).json({
```

```
            message: "Error retrieving events"
        });
    }
});

// POST: Create a new event
app.post("/api/events", async (req, res) => {
    try {
        const {
            eventId,
            title,
            venue,
            date,
            fees,
            organizerName
        } = req.body;

        if (
            !Number.isInteger(eventId) ||
            eventId < 1 ||
            !title?.trim() ||
            !venue?.trim() ||
            !date ||
            !Number.isFinite(fees) ||
            fees < 0 ||
            !organizerName?.trim()
        ) {
```

```
            return res.status(400).json({
                message: "Please enter valid event details"
            });
        }

        const event = new Event({
            eventId,
            title: title.trim(),
            venue: venue.trim(),
            date,
            fees,
            organizerName: organizerName.trim()
        });

        await event.save();

        res.status(201).json({
            message: "Event created successfully",
            event
        });

    } catch (err) {
        if (err.code === 11000) {
            return res.status(409).json({
                message: "Event ID already exists"
            });
        }
```

```
        res.status(500).json({
            message: "Error creating event"
        });
    }
});

// DELETE: Cancel an event by event ID
app.delete("/api/events/:eventId", async (req, res) => {
    try {
        const eventId = Number(req.params.eventId);

        if (!Number.isInteger(eventId) || eventId < 1) {
            return res.status(400).json({
                message: "Invalid event ID"
            });
        }

        const event = await Event.findOneAndDelete({
            eventId: eventId
        });

        if (!event) {
            return res.status(404).json({
                message: "Event not found"
            });
        }
```

```
        res.json({
            message: "Event cancelled successfully"
        });

    } catch (err) {
        res.status(500).json({
            message: "Error cancelling event"
        });
    }
});

// Start server after MongoDB connects
mongoose.connection.once("open", () => {
    app.listen(3000, () => {
        console.log("Server running at http://localhost:3000");
    });
});
```

- **Input(index.html):-**

```
<!DOCTYPE html>
<html ng-app="eventApp">

<head>
    <title>Event Management System</title>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
    <link rel="stylesheet" href="style.css">
</head>
```

```
<body ng-controller="eventController">

    <div class="container">

        <h1>Event Management System</h1>

        <!-- Add Event Form -->
        <h2>Create New Event</h2>

        <form ng-submit="addEvent()" novalidate>

            <label>Event ID:</label>
            <input type="number"
                   ng-model="form.eventId"
                   min="1"
                   step="1"
                   required>

            <label>Event Title:</label>
            <input type="text"
                   ng-model="form.title"
                   required>

            <label>Venue:</label>
            <input type="text"
                   ng-model="form.venue"
```

```
            required>

    <label>Date:</label>
    <input type="date"
            ng-model="form.date"
            required>

    <label>Fees:</label>
    <input type="number"
            ng-model="form.fees"
            min="0"
            step="0.01"
            required>

    <label>Organizer Name:</label>
    <input type="text"
            ng-model="form.organizerName"
            required>

    <button type="submit">Add Event</button>

</form>

<!-- Messages -->
<p class="success" ng-if="message">
    {{ message }}
</p>
```

```
<p class="error" ng-if="errorMessage">
   {{ errorMessage }}
</p>
<!-- Display Events -->
<h2>All Events</h2>

<button ng-click="getEvents()">
   Refresh Events
</button>

<table>
   <thead>
      <tr>
         <th>Event ID</th>
         <th>Title</th>
         <th>Venue</th>
         <th>Date</th>
         <th>Fees</th>
         <th>Organizer</th>
         <th>Action</th>
      </tr>
   </thead>

   <tbody>
      <tr ng-repeat="event in events">
         <td>{{ event.eventId }}</td>
```

```
            <td>{{ event.title }}</td>
            <td>{{ event.venue }}</td>
            <td>{{ event.date | date:'dd-MM-yyyy' }}</td>
            <td>{{ event.fees }}</td>
            <td>{{ event.organizerName }}</td>

            <td>
              <button class="cancel-btn"
                      ng-click="cancelEvent(event.eventId)">
                Cancel Event
              </button>
            </td>
          </tr>

          <tr ng-if="events.length === 0">
            <td colspan="7">No events available.</td>
          </tr>
        </tbody>
      </table>

  </div>

  <script src="app.js"></script>

</body>
</html>
```

- **Input(app.js):-**

```
var app = angular.module("eventApp", []);

app.controller("eventController", function($scope, $http) {

    $scope.events = [];
    $scope.form = {};
    $scope.message = "";
    $scope.errorMessage = "";

    // GET: Retrieve all events
    $scope.getEvents = function() {

        $http.get("/api/events")
            .then(function(response) {

                $scope.events = response.data;
                $scope.errorMessage = "";

            })
            .catch(function() {

                $scope.errorMessage = "Unable to load events.";

            });
    };

    // POST: Create a new event
```

```
$scope.addEvent = function() {

    $scope.message = "";
    $scope.errorMessage = "";

    var eventId = Number($scope.form.eventId);
    var fees = Number($scope.form.fees);

    if (
        !Number.isInteger(eventId) ||
        eventId < 1 ||
        !$scope.form.title ||
        !$scope.form.title.trim() ||
        !$scope.form.venue ||
        !$scope.form.venue.trim() ||
        !$scope.form.organizerName ||
        !$scope.form.organizerName.trim() ||
        !$scope.form.date ||
        !Number.isFinite(fees) ||
        fees < 0
    ) {
        $scope.errorMessage =
            "Please enter valid details in all fields.";
        return;
    }

    var eventData = {
```

```
        eventId: eventId,
        title: $scope.form.title.trim(),
        venue: $scope.form.venue.trim(),
        date: $scope.form.date,
        fees: fees,
        organizerName: $scope.form.organizerName.trim()
    };

    $http.post("/api/events", eventData)
        .then(function(response) {

            $scope.message = response.data.message;
            $scope.form = {};

            $scope.getEvents();

        })
        .catch(function(error) {

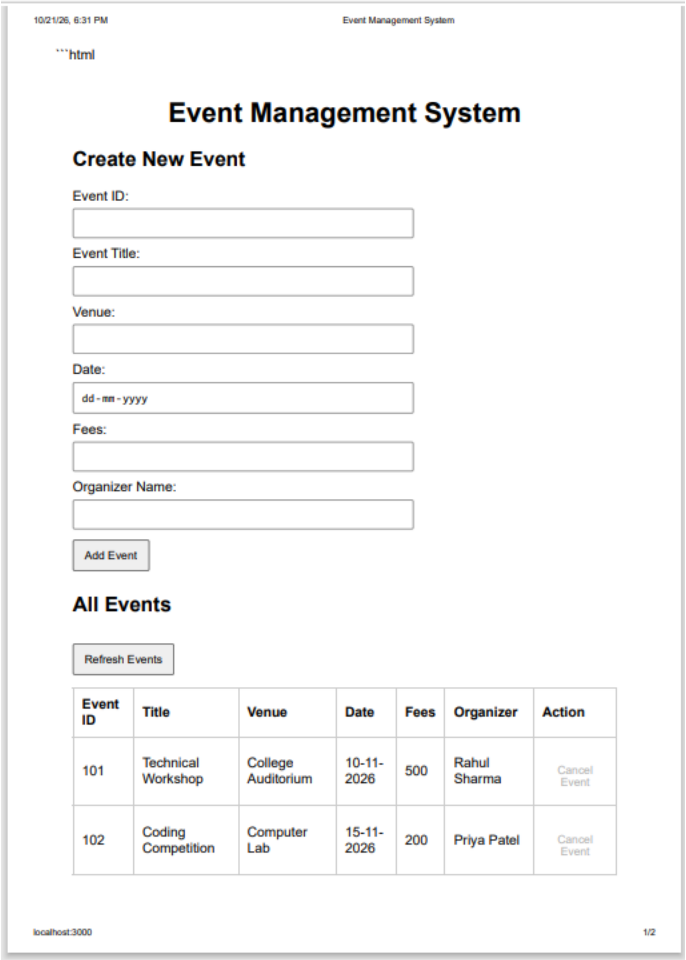
            $scope.errorMessage =
                error.data && error.data.message
                    ? error.data.message
                    : "Unable to create event.";

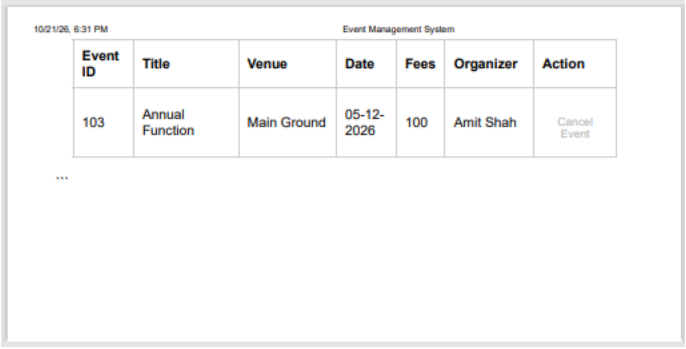
        });
};
```

```
    // DELETE: Cancel an event
    $scope.cancelEvent = function(eventId) {

        if (!confirm("Are you sure you want to cancel this event?")) {
            return;
        }
        $http.delete("/api/events/" + eventId)
            .then(function(response) {
                $scope.message = response.data.message;
                $scope.errorMessage = "";
                $scope.getEvents();
            })
            .catch(function(error) {
                $scope.errorMessage =
                    error.data && error.data.message
                        ? error.data.message
                        : "Unable to cancel event.";
            });
    };
    // Load events when page opens
    $scope.getEvents();
});
```

- **Output :-**


10/21/26, 6:31 PM
Event Management System
```html
Event Management System
Create New Event
Event ID:
Event Title:
Venue:
Date:
dd - mm - yyyy
Fees:
Organizer Name:
Add Event
All Events
Refresh Events
Event ID
Title
Venue
Date
Fees
Organizer
Action
101
Technical Workshop
College Auditorium
10-11-2026
500
Rahul Sharma
Cancel Event
102
Coding Competition
Computer Lab
15-11-2026
200
Priya Patel
Cancel Event
localhost:3000
1/2


10/21/26, 6:31 PM
Event Management System
Event ID
Title
Venue
Date
Fees
Organizer
Action
103
Annual Function
Main Ground
05-12-2026
100
Amit Shah
Cancel Event
```